

THEORY OF COMPUTATION

Unit I: Foundations

Alphabets, Strings, Languages, Regular Expressions & Context-Free Grammars — explained the simple way, for everyone.

Simple language • Everyday examples • Step-by-step



SAGE
UNIVERSITY
—BHOPAL—

What we'll cover

Unit I — in plain, simple steps

1

1. Alphabets, Strings & Languages

The building blocks — letters, words, and sets of words

3

3. Finite Automata (basics)

A simple machine that checks strings

5

5. Parse Trees

A picture of how a sentence was built

2

2. Regular Expressions

A short pattern that describes a whole language

4

4. Context-Free Grammars

Rules to generate valid sentences

6

6. Ambiguity & Regular Grammar

When one sentence has two meanings



SAGE
UNIVERSITY
—BHOPAL—

SECTION 1 OF 6

Alphabets, Strings & Languages

The ABC of this whole subject — don't worry, it's genuinely simple



SAGE
UNIVERSITY
— Bhopal —

Alphabet (Σ)

Think of it like this: It's just like the keyboard buttons you're allowed to use — nothing fancy.

In simple words: A finite set of symbols (letters) that we're allowed to use to build words.

- Written as the Greek letter Σ (sigma)
- It must be finite (limited, countable) — not infinite
- Symbols can be anything: letters, digits, even emojis!

Example: $\Sigma = \{0, 1\} \rightarrow$ only two symbols allowed, like a light switch: ON or OFF



SAGE
UNIVERSITY
—BHOPAL—

String

Think of it like this: A string is just a WORD — you pick letters from the alphabet and place them in a row.

In simple words: A finite sequence of symbols, all taken from some alphabet Σ .

- Order matters — 'ab' and 'ba' are different strings
- A string can repeat symbols as many times as needed
- Even ZERO letters is allowed — that's called the empty string

Example: If $\Sigma = \{a, b\}$ → strings like: a, ab, aab, abba, bbb ...



SAGE
UNIVERSITY
—BHOPAL—

QUICK CHECK

Which of these is NOT a valid string over $\Sigma = \{0, 1\}$?

A 101

B 000111

C 0

D 012

Answer: D — 012 because '2' is not in the alphabet $\{0,1\}$. Every symbol in a string must come from Σ .



SAGE
UNIVERSITY
—BHOPAL—

String Operations

Think of it like this: Think of a string like a chain of beads — you can measure it, join two chains, or flip it around.

In simple words: There are simple things we can DO to strings.

- Length $|w|$ → how many symbols in the string ($|abc| = 3$)
- Concatenation → joining two strings end to end ('ab' + 'cd' = 'abcd')
- Reversal w^R → flip the string backward ('cat' → 'tac')

Example: $|hello| = 5$, 'good' + 'bye' = 'goodbye', $\text{reverse}('abc') = 'cba'$



SAGE
UNIVERSITY
—BHOPAL—

SPECIAL CASES

Empty String, Σ^* and Σ^+

Think of it like this: Σ^* is like 'every possible message you could EVER type' using your keyboard — even a blank message.

In simple words: Some special, commonly-used symbols you'll see everywhere.

- ϵ (epsilon) $\rightarrow$ the empty string — zero symbols, length 0
- Σ^* $\rightarrow$ the SET of every possible string (including ϵ) using letters from Σ
- Σ^+ $\rightarrow$ same as Σ^* , but WITHOUT the empty string

Example: If $\Sigma = \{a\} \rightarrow \Sigma^* = \{ \epsilon, a, aa, aaa, aaaa, \dots \}$ (goes on forever!)



SAGE
UNIVERSITY
— BHOPAL —

Language

Think of it like this: If Σ^* is 'every possible word you could ever make', a Language is just the ones you actually SELECT — like a playlist chosen out of every song ever made.

In simple words: A language is simply any set of strings — i.e., any subset of Σ^* .

- A language can be finite ({cat, dog, bat}) or infinite (all even-length strings)
- It doesn't have to 'mean' anything in English — it's just a chosen collection of strings
- Even the empty set $\{ \}$ counts as a valid language

Example: $L = \{ \text{strings over } \{0,1\} \text{ that start with } 1 \} = \{1, 10, 11, 100, 101, \dots\}$



SAGE
UNIVERSITY
—BHOPAL—

Operations on Languages

Think of it like this: If languages are playlists, we can merge two playlists, play one after another, or repeat a playlist endlessly.

In simple words: Just like sets, we can combine languages using simple operations.

- Union ($L1 \cup L2$) $\rightarrow$ strings from EITHER language
- Concatenation ($L1.L2$) $\rightarrow$ take one string from L1, glue a string from L2 after it
- Kleene Star (L^*) $\rightarrow$ glue ANY number of strings from L together (0, 1, 2, 3... times)

Example: If $L = \{a, b\}$, then $L^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$



SAGE
UNIVERSITY
—BHOPAL—

WHY WE NEED THIS

Finite Representation of Languages

Think of it like this: A language can have INFINITE strings — you obviously can't write a list that never ends! So we need a short 'recipe' instead.

In simple words: A finite (short) rule or description that represents a language, even an infinite one.

- Regular Expressions → a compact pattern/formula
- Grammars → a set of rewriting rules
- Automata (machines) → a step-by-step checker

Example: Instead of listing every even number forever, we just say: 'numbers ending in 0,2,4,6,8' — one short rule, infinite numbers covered!



SAGE
UNIVERSITY
—BHOPAL—

SECTION 2 OF 6

Regular Expressions

A tiny pattern that can describe a HUGE (even infinite) set of strings



SAGE
UNIVERSITY
—BHOPAL—

What is a Regular Expression?

Think of it like this: Same as when you search WhatsApp/Google with a pattern — you're telling it the SHAPE of what you want, not every possible answer one by one.

In simple words: A short formula built from symbols and a few operators, used to describe a language in one line.

- Written using the alphabet symbols + 3 special operators
- Every regular expression represents exactly one language
- Used everywhere — search boxes, form validation (email/phone checks), text editors

Example: The pattern for 'a valid 10-digit phone number' is basically a regular expression!



SAGE
UNIVERSITY
— BHOPAL —

RE Operators & Order

Think of it like this: Just like Maths has BODMAS (which operation to do first), Regular Expressions have their own order of priority.

In simple words: Three simple operators build every regular expression.

- * (star) → repeat ZERO or more times — done FIRST
- . (concatenation) → just place symbols one after another — done SECOND
- | (union / OR) → either this OR that — done LAST

Example: `a|bc*` means a OR (b followed by zero-or-more c) → so 'a', 'b', 'bc', 'bcc' all match



SAGE
UNIVERSITY
—BHOPAL—

Common Regular Expression Patterns

Pattern	In Plain English
<code>a*</code>	zero or more 'a' — even nothing at all (ϵ , a, aa, aaa...)
<code>a+</code>	one or more 'a' — at least one required (a, aa, aaa...)
<code>a b</code>	either 'a' OR 'b' — just one of the two
<code>(ab)*</code>	the whole block 'ab' repeated any number of times
<code>(a b)*</code>	any string made only from a's and b's, in any order
<code>0(0 1)*</code>	strings starting with 0, followed by any 0s and 1s



SAGE
UNIVERSITY
—BHOPAL—

From Pattern → to Language

Think of it like this: Reading a regex is like reading a recipe — follow it step by step and you'll know exactly what strings it 'cooks'.

In simple words: Every regular expression quietly describes a full set of matching strings — let's decode one.

- Pattern: `(a|b)*abb`
- Meaning: any mix of a's/b's ... followed by exactly 'abb' at the end
- So it matches strings that END with 'abb'

Example: Matches: `abb, aabb, babb, ababb` | Does NOT match: `abba, ab, ba`



SAGE
UNIVERSITY
—BHOPAL—

SECTION 3 OF 6

Finite Automata — the Basics

Meet the simplest possible 'string-checking machine'



SAGE
UNIVERSITY
— Bhopal —

What is a Finite Automaton?

Think of it like this: Imagine a video game character standing on a tile (state). Every letter you type moves the character to another tile. At the end, we check: did it land on a 'winning tile'?

In simple words: A simple machine with a limited (finite) memory, used to check whether a string belongs to a language.

- It reads the input string one symbol at a time, left to right
- It has NO extra memory — only 'which state it's currently in'
- At the end, it either ACCEPTS or REJECTS the string

Example: Think of a turnstile at a metro station — it only remembers 'locked' or 'unlocked', nothing more!



Anatomy of a Finite Automaton

Think of it like this: Like a simple board game: you have positions (states), a starting square, dice-roll moves (transitions), and a few 'winning squares' (final states).

In simple words: Every finite automaton is made up of just these simple pieces.

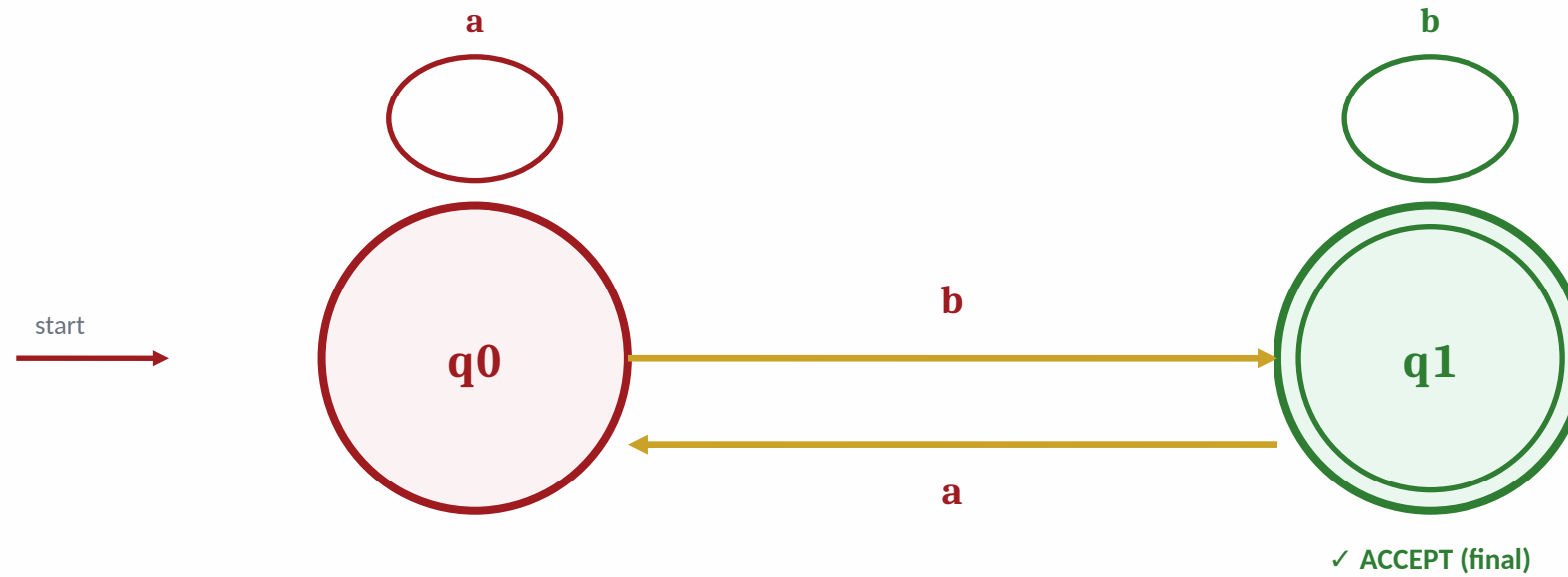
- States → the 'positions' the machine can be in (drawn as circles)
- Start state → where the machine begins (has an arrow pointing in)
- Transitions → rules for moving from one state to another on reading a symbol
- Final / accepting states → 'winning' states (drawn as double circles)

Example: If the machine ends on a double-circle state → string ACCEPTED. Otherwise → REJECTED.



Picture of a Finite Automaton

A machine that reads letters one by one and hops between 'states' (bubbles)



Simple example: for the string "ab" — start at q_0 , read 'a' (stay q_0), read 'b' (jump to q_1). We stop at a final state → string ACCEPTED!



SAGE
UNIVERSITY
—BHOPAL—

SECTION 4 OF 6

Context-Free Grammars (CFG)

A recipe book of rules that 'generates' valid sentences



SAGE
UNIVERSITY
—Bhopal—

What is a CFG?

Think of it like this: Think of language rules you learned in school: 'Sentence = Subject + Verb + Object'. A CFG is exactly that, but for formal languages.

In simple words: A CFG is a set of rewriting rules used to GENERATE all valid strings of a language.

- Written as 4 parts: $G = (V, T, P, S)$
- $V \rightarrow$ Variables, $T \rightarrow$ Terminals, $P \rightarrow$ Productions, $S \rightarrow$ Start symbol
- Why needed? Regex CAN'T describe things like 'equal a's and b's' — CFGs can!

Example: $S \rightarrow aSb \mid ab$ generates: ab, aabb, aaabbb, ...



SAGE
UNIVERSITY
—BHOPAL—

A Simple CFG Example

Think of it like this: It's like a Mad Libs game — you keep replacing a blank (variable) with words (terminals or more blanks) until no blanks are left.

In simple words: Grammar for 'equal number of a's followed by equal number of b's':

- $S \rightarrow aSb$ (add one 'a' in front, one 'b' at the back)
- $S \rightarrow ab$ (the simplest valid ending)

Example: Start with $S \rightarrow aSb \rightarrow aaSbb \rightarrow aaabbbb$ (replace last S with ab) = aaabbbb ✓



SAGE
UNIVERSITY
—BHOPAL—

Sentential Forms

Think of it like this: It's just a 'work-in-progress' sentence — like a half-cooked dish, some parts are ready (terminals), some are still cooking (variables).

In simple words: Any string we get in the MIDDLE of a derivation — it can have a mix of terminals AND variables.

- We start from S (just the start symbol)
- Each step replaces ONE variable using a production rule
- We stop when NO variables are left — that final string is the result

Example: $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaabbb$ → each of these is a sentential form



SAGE
UNIVERSITY
—BHOPAL—

Leftmost Derivation

Think of it like this: Imagine reading left to right and fixing (replacing) whichever blank you see FIRST.

In simple words: A derivation where, at every step, we always replace the LEFTMOST variable first.

- Grammar: $S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow b$
- $S \Rightarrow AB \Rightarrow aB$ (replace LEFTMOST variable A) $\Rightarrow ab$

Example: Leftmost derivation of 'ab': $S \Rightarrow AB \Rightarrow aB \Rightarrow ab$



SAGE
UNIVERSITY
—BHOPAL—

Rightmost Derivation

Think of it like this: Same game, but now we fix the blank that's furthest to the RIGHT first.

In simple words: A derivation where, at every step, we always replace the RIGHTMOST variable first.

- Grammar: $S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow b$
- $S \Rightarrow AB \Rightarrow Ab$ (replace RIGHTMOST variable B) $\Rightarrow ab$

Example: Rightmost derivation of 'ab': $S \Rightarrow AB \Rightarrow Ab \Rightarrow ab$ (same word, different step order)



QUICK CHECK

In $S \Rightarrow AB$, if we replace A before B, is that leftmost or rightmost?

- A Leftmost derivation
- B Rightmost derivation
- C Both are the same thing
- D Neither — this is invalid

Answer: A — Leftmost derivation because A is positioned to the LEFT of B in the string 'AB', and we replaced it first.



SAGE
UNIVERSITY
— BHOPAL —

Language of a CFG — $L(G)$

Think of it like this: It's simply the FULL playlist of every song (string) this grammar's rules are able to generate.

In simple words: $L(G)$ = the set of ALL strings (made only of terminals) that can be derived starting from S .

- Only strings with ZERO variables left count as 'in the language'
- A grammar can generate infinitely many strings, using a finite set of rules
- Two different grammars can even generate the exact same language

Example: For $S \rightarrow aSb \mid ab$: $L(G) = \{ ab, aabb, aaabbb, \dots \}$ — equal a's and b's



SAGE
UNIVERSITY
—BHOPAL—

SECTION 5 OF 6

Parse Trees

A picture that shows HOW a sentence was actually built



SAGE
UNIVERSITY
—Bhopal—

What is a Parse Tree?

Think of it like this: It's basically a family tree — but instead of grandparents and grandchildren, it's grammar rules and the words they produced.

In simple words: A tree diagram that shows how a string was derived from the start symbol, step by step, using the grammar's rules.

- The ROOT (top) of the tree is always the start symbol S
- Each internal node is a variable that got expanded using a rule
- The LEAVES (bottom-most nodes) spell out the final string, left to right

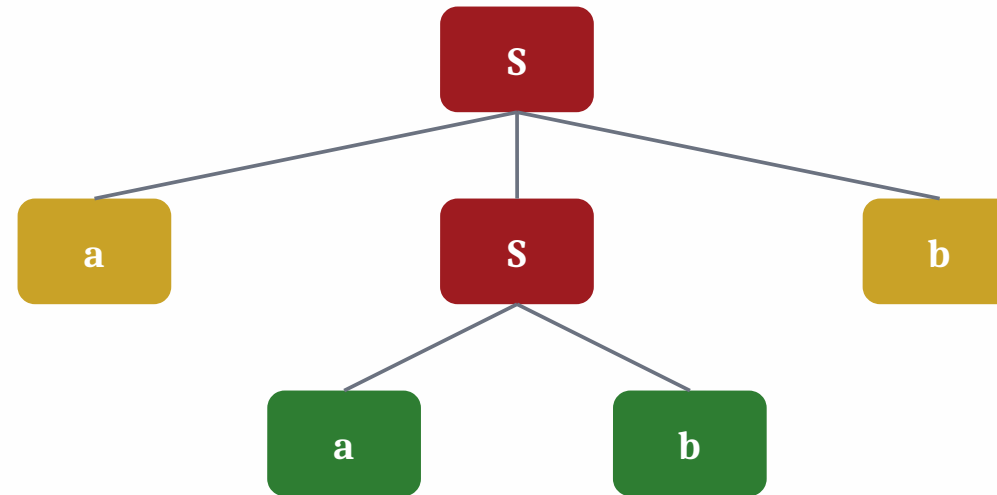
Example: Also called a 'derivation tree' — same thing, just another name!



SAGE
UNIVERSITY
— BHOPAL —

Picture of a Parse Tree

Grammar: $S \rightarrow aSb \mid ab$ Building the word "aabb" step by step



Read the bottom leaves left → right:

a a b b

= our word "aabb" ✓

Simple idea: the tree just shows HOW the grammar built the word — like a family tree, but for sentences instead of people.



SAGE
UNIVERSITY
—BHOPAL—

Parse Trees $\leftrightarrow$ Derivations

Think of it like this: Think of a tree as the FINAL PHOTO, and a derivation as a VIDEO of how we got there. Different videos (leftmost/rightmost) can end in the exact same photo!

In simple words: One parse tree can correspond to MULTIPLE derivations — the tree just doesn't record the ORDER in which variables were replaced.

- Leftmost and rightmost derivation of the SAME string $\rightarrow$ SAME tree, different step order
- The tree shows structure; the derivation shows the sequence of steps
- If a string has only ONE possible tree $\rightarrow$ grammar is unambiguous for that string

Example: Both $S \Rightarrow AB \Rightarrow aB \Rightarrow ab$ AND $S \Rightarrow AB \Rightarrow Ab \Rightarrow ab$ draw the exact same tree.



SAGE
UNIVERSITY
—BHOPAL—

SECTION 6 OF 6

Parsing, Ambiguity & Regular Grammar

When one sentence can be understood in two different ways



SAGE
UNIVERSITY
—Bhopal—

Ambiguity in Grammar

Think of it like this: Just like the English sentence 'I saw a man with a telescope' can mean two different things — grammars can also be confusing like that!

In simple words: A grammar is **AMBIGUOUS** if at least one string in its language has **MORE THAN ONE** parse tree (or leftmost derivation).

- One string → two different structures → two different meanings
- This is a real **PROBLEM** for programming languages (compilers get confused!)
- We often try to **REWRITE** an ambiguous grammar into an unambiguous one

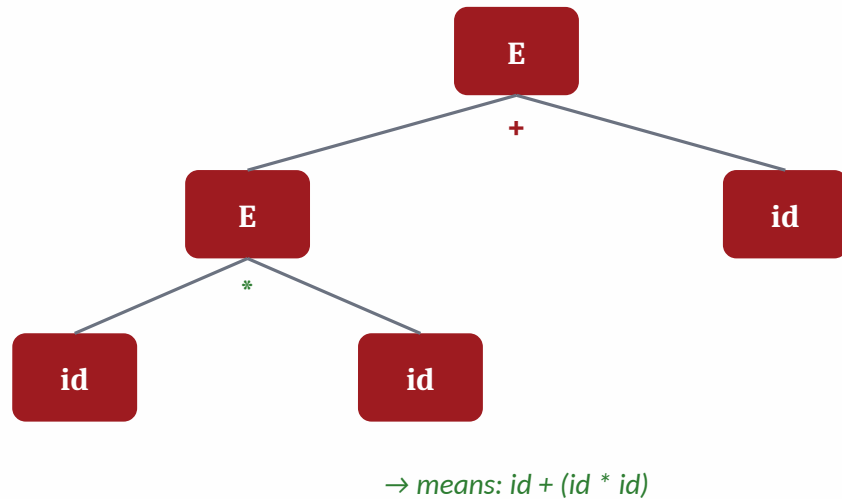
Example: Same word, two trees = ambiguous grammar. Compilers must pick just **ONE** meaning.



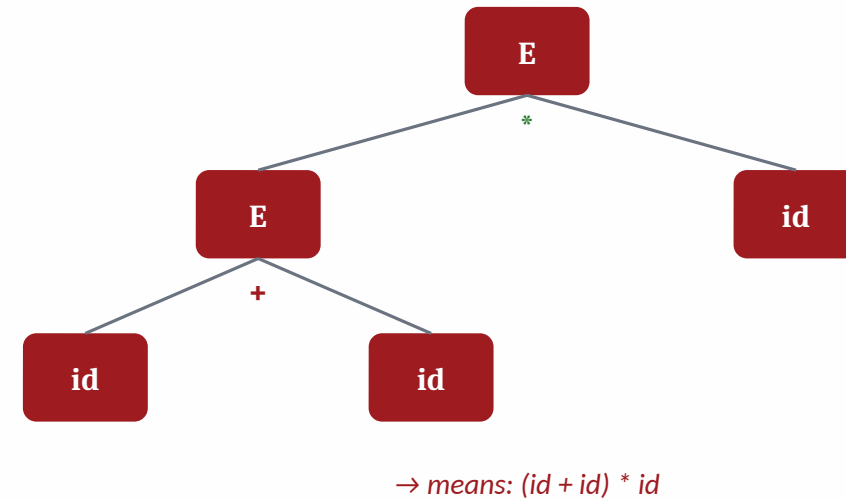
Ambiguity = 2 different meanings, 1 sentence

Grammar: $E \rightarrow E+E \mid E^*E \mid id$ Word: $id + id * id$

Tree 1



Tree 2



Simple idea: Same sentence "id + id * id", but two different trees give two different answers — just like "I saw a man with a telescope" could mean YOU had the telescope, or HE did. When a grammar can build more than one tree for the same word, we call it ambiguous.



SAGE
UNIVERSITY
— BHOPAL —

Regular Grammar

Think of it like this: It's the 'starter version' of a grammar — simple enough that it matches exactly what a Finite Automaton (and Regular Expressions) can do.

In simple words: A CFG where every production is in a very restricted, simple form — either right-linear or left-linear.

- Right-linear: $A \rightarrow aB$ or $A \rightarrow a$ (terminal, then AT MOST one variable)
- Left-linear: $A \rightarrow Ba$ or $A \rightarrow a$ (variable comes first, then a terminal)
- Same power as Regular Expressions and Finite Automata

Example: $S \rightarrow aS \mid b$ generates: $b, ab, aab, aaab, \dots$ (any number of a's, then one b)



Unit I — Quick Recap

Everything we covered, in one glance

- ✓ Alphabet = allowed symbols; String = a word made of them
- ✓ Regular Expression = a short pattern describing a language
- ✓ CFG (V,T,P,S) = rules that GENERATE valid strings
- ✓ Leftmost/Rightmost derivation = order of replacing variables
- ✓ Ambiguous grammar = one string, more than one tree
- ✓ Language = a chosen SET of strings
- ✓ Finite Automaton = a simple machine that checks strings
- ✓ Sentential form = a work-in-progress string mid-derivation
- ✓ Parse Tree = a picture of how a string was built
- ✓ Regular grammar = simplest CFG, same power as FA/RE

Next up → Unit II: Finite Automata (DFA, NFA) in detail



SAGE
UNIVERSITY
— BHOPAL —